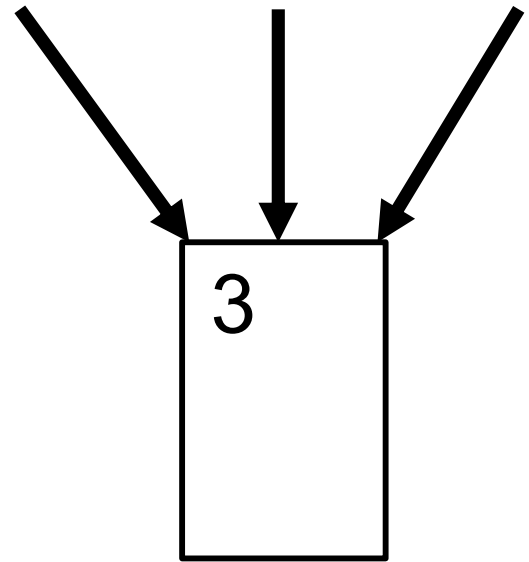
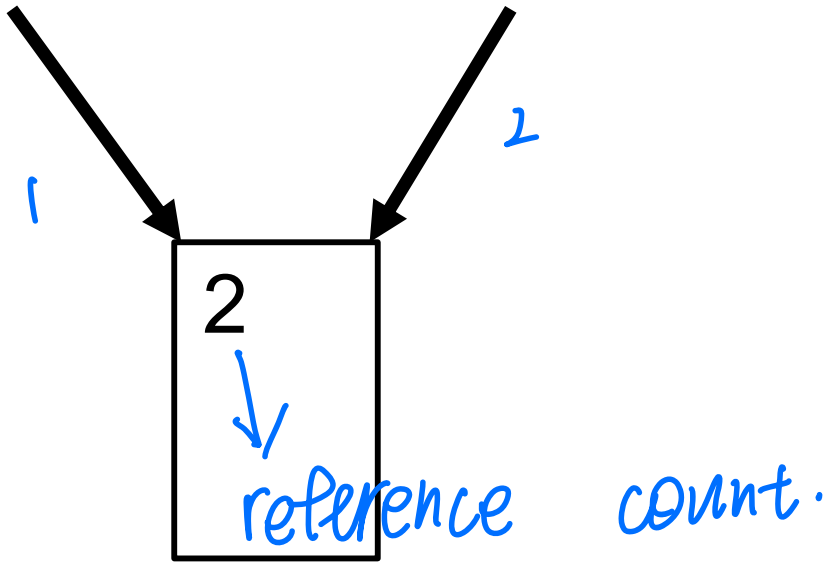
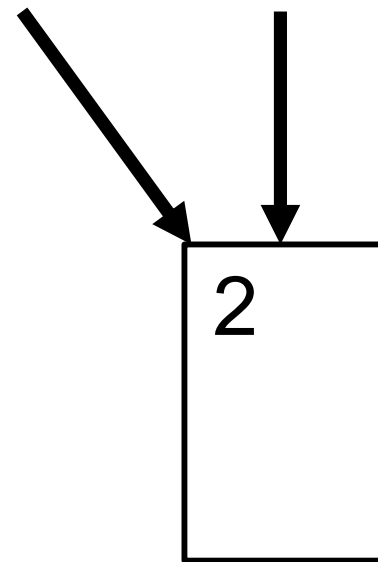
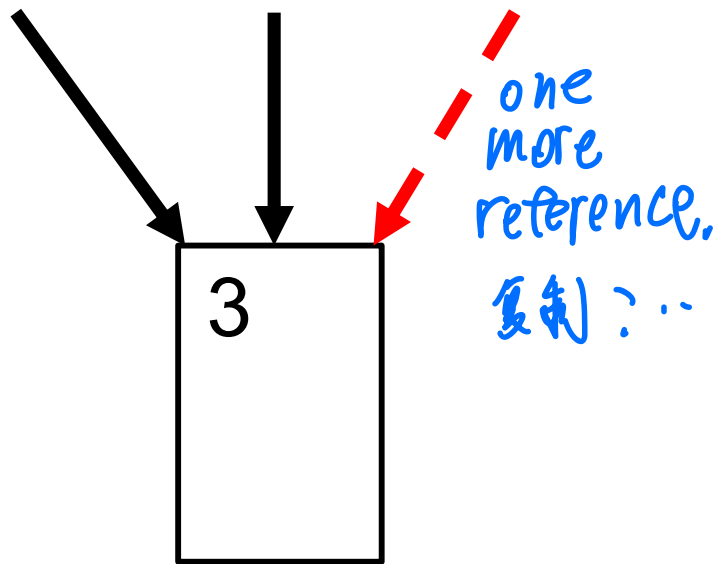
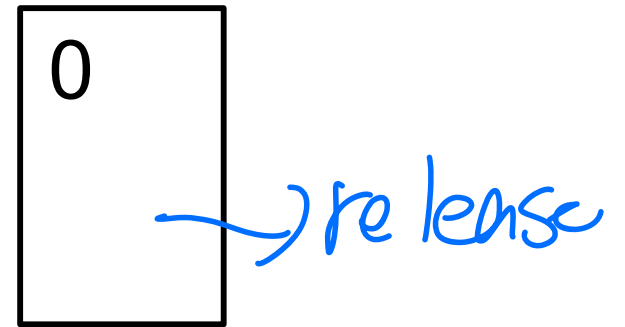
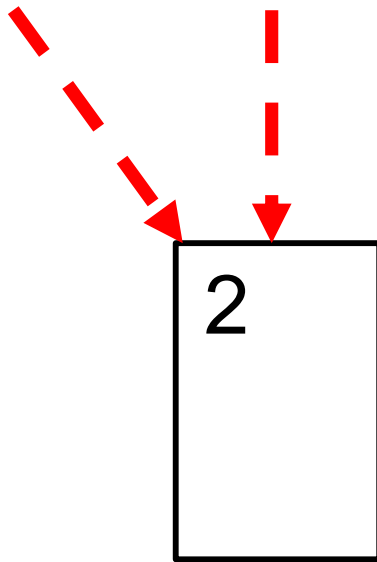


Garbage Collection

- ❑ Modern programming languages provide garbage collection mechanisms for reclaiming the memory locations that are no longer used by programs
- ❑ One of the commonly used technique in centralised system is reference counting
 - each object keeps a **reference count**
 - when the reference to an object is **copied**, the reference count of the object is incremented
 - when the reference to an object is **deleted**, the reference count of the object is decremented
 - when the reference count **becomes 0**, the object can be garbage collected

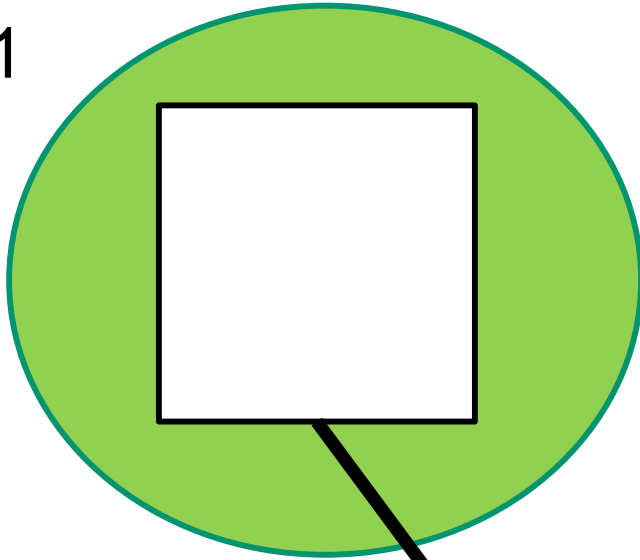




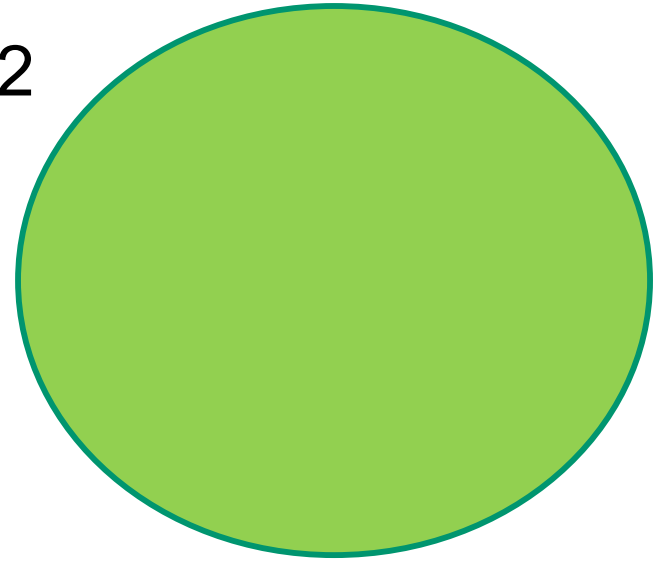


- problems with reference counting when applied in a distributed system
 - object might be garbage collected ^{it's} prematurely
 - The problem is due to the messages for incrementing and decrementing count are received out of order

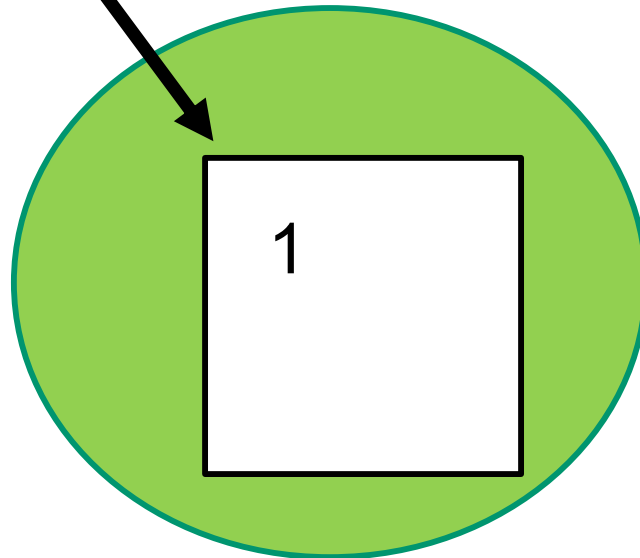
M1

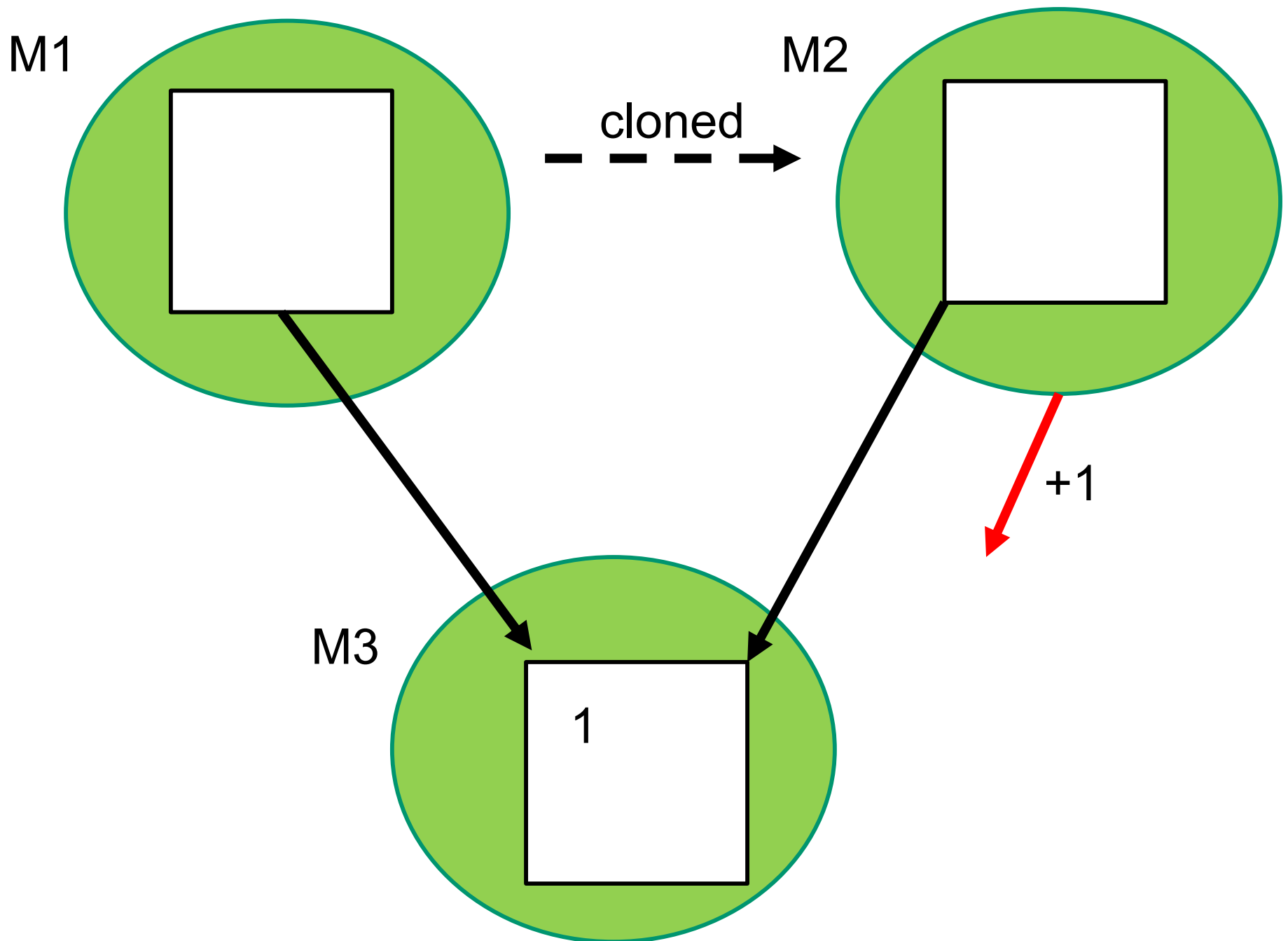


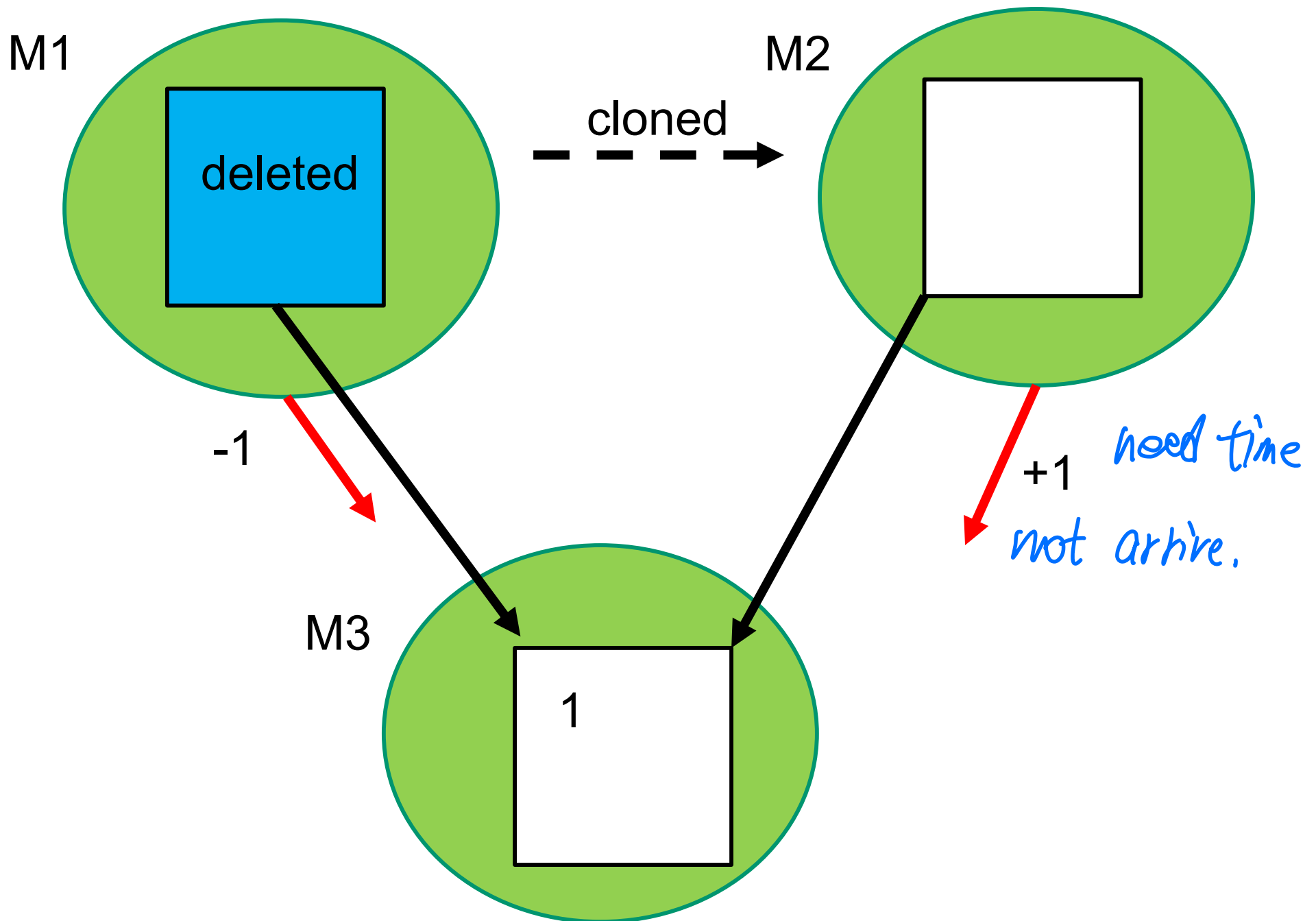
M2

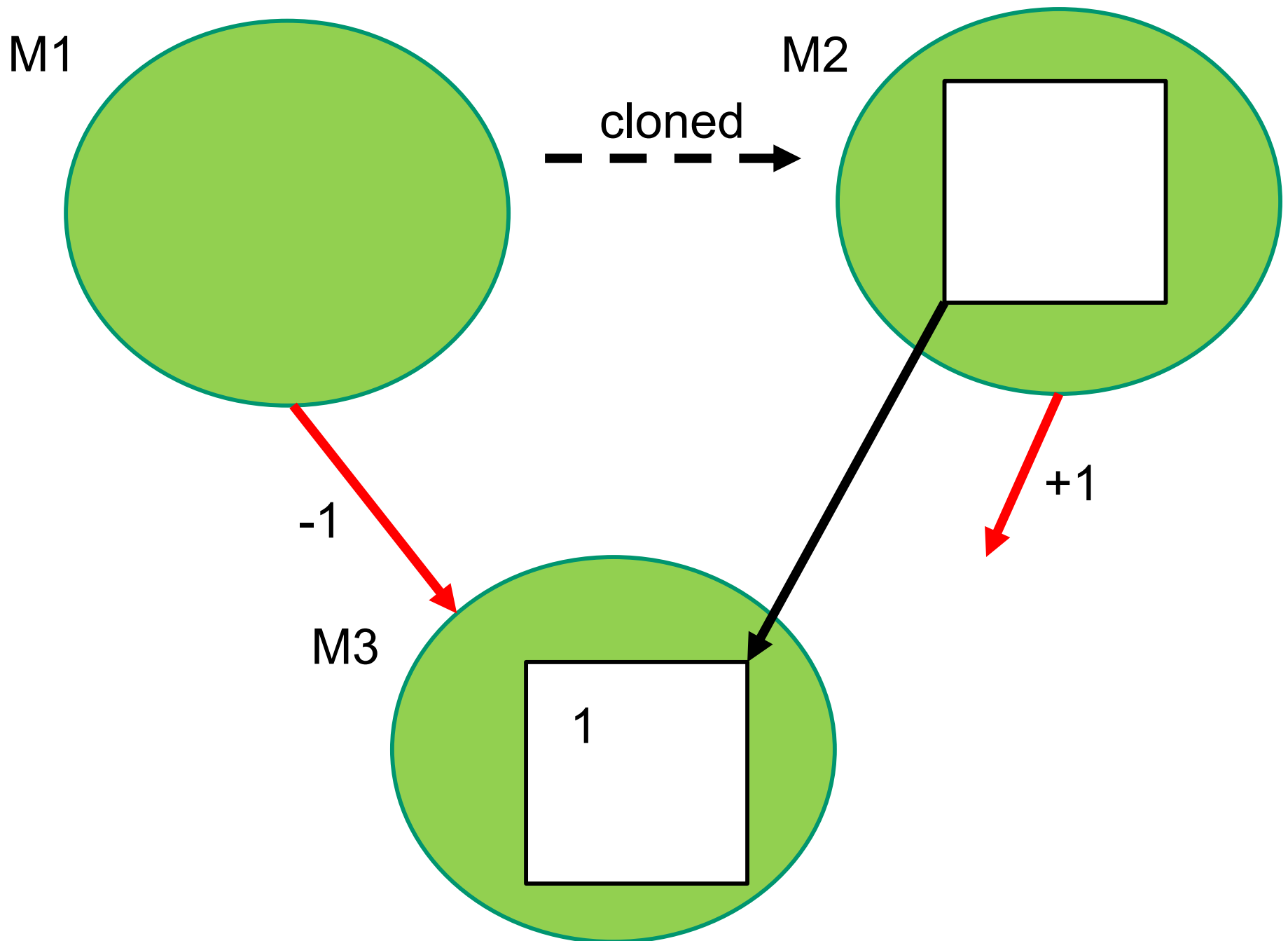


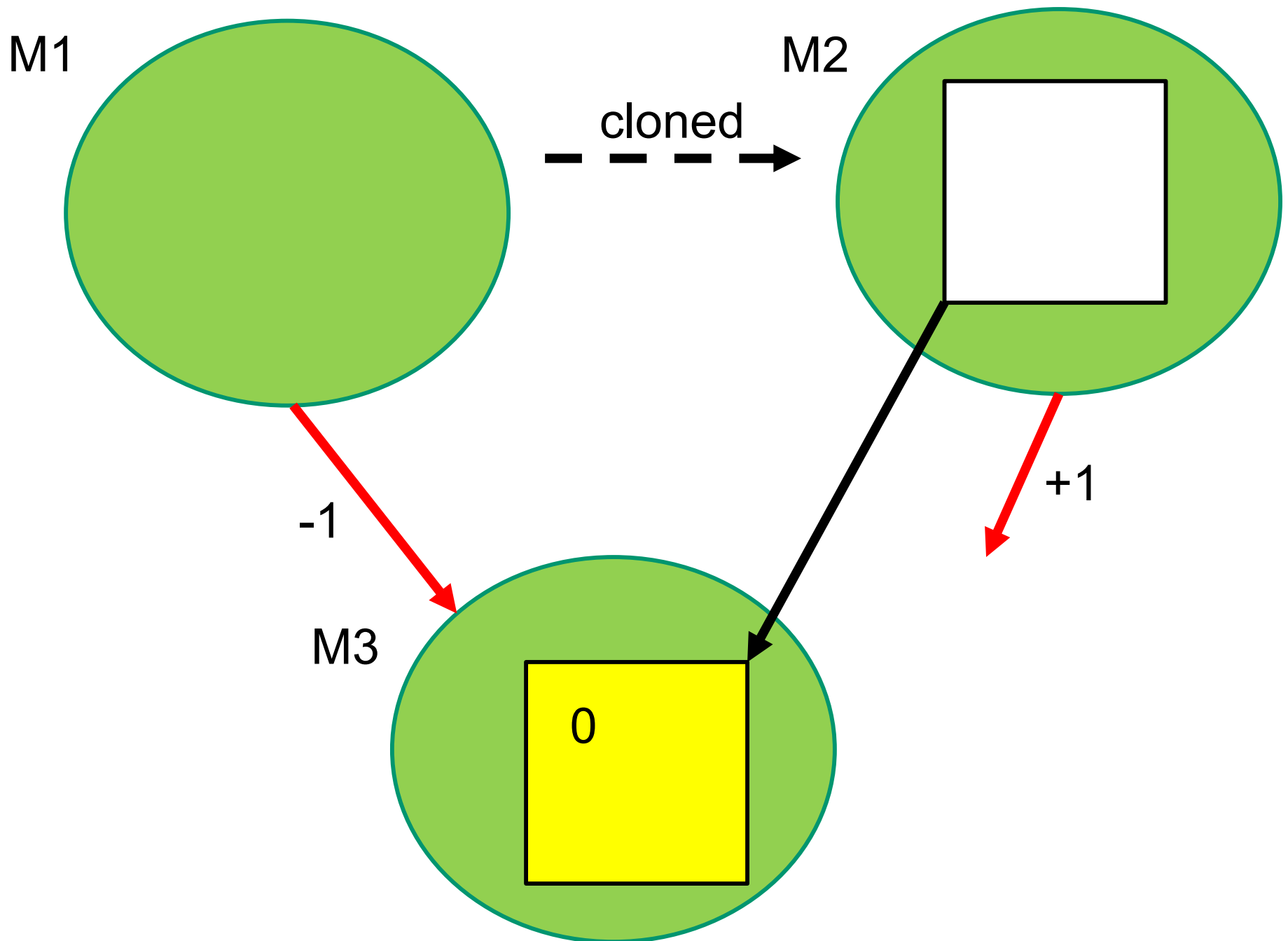
M3

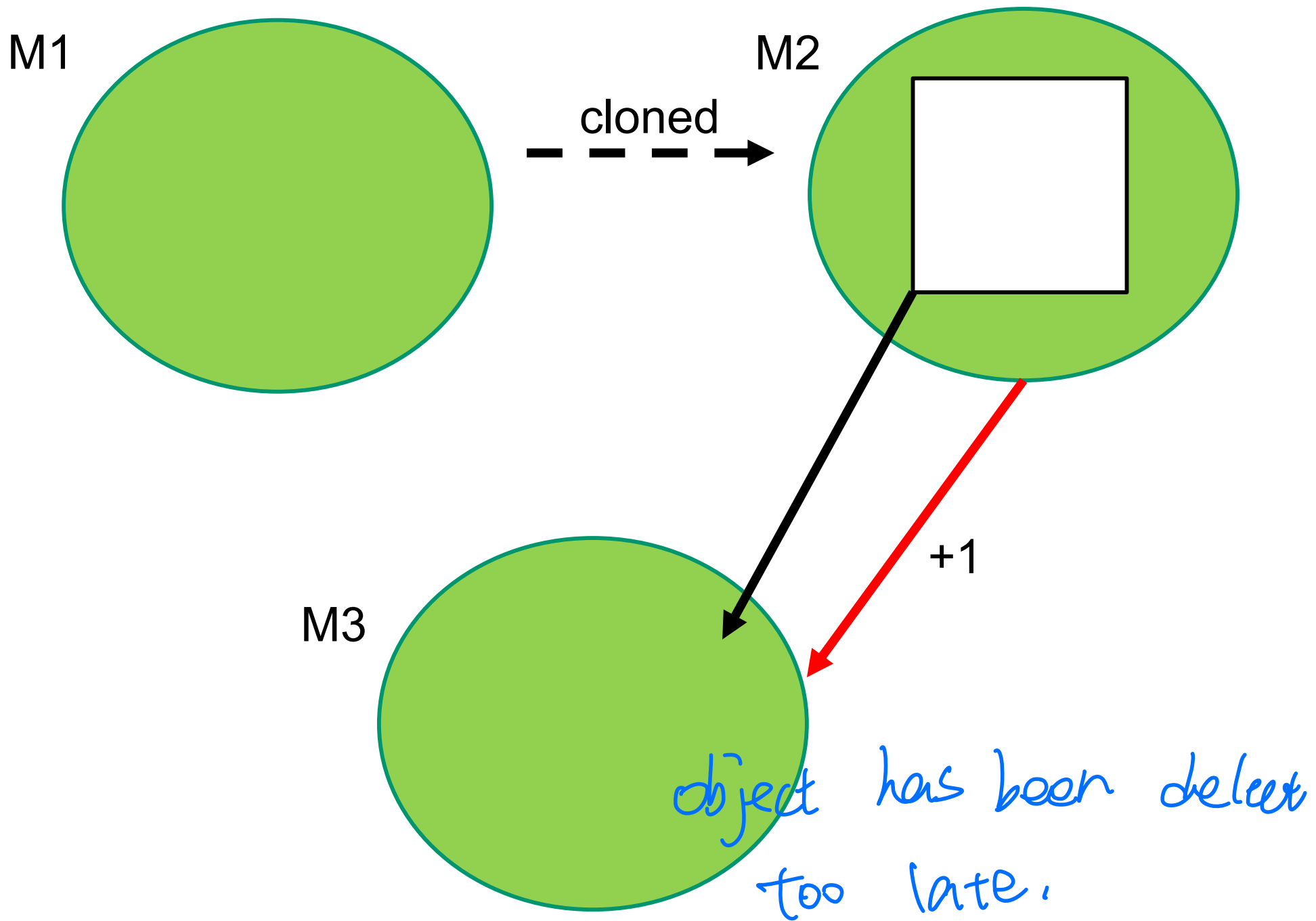






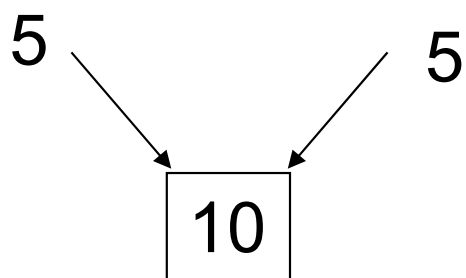
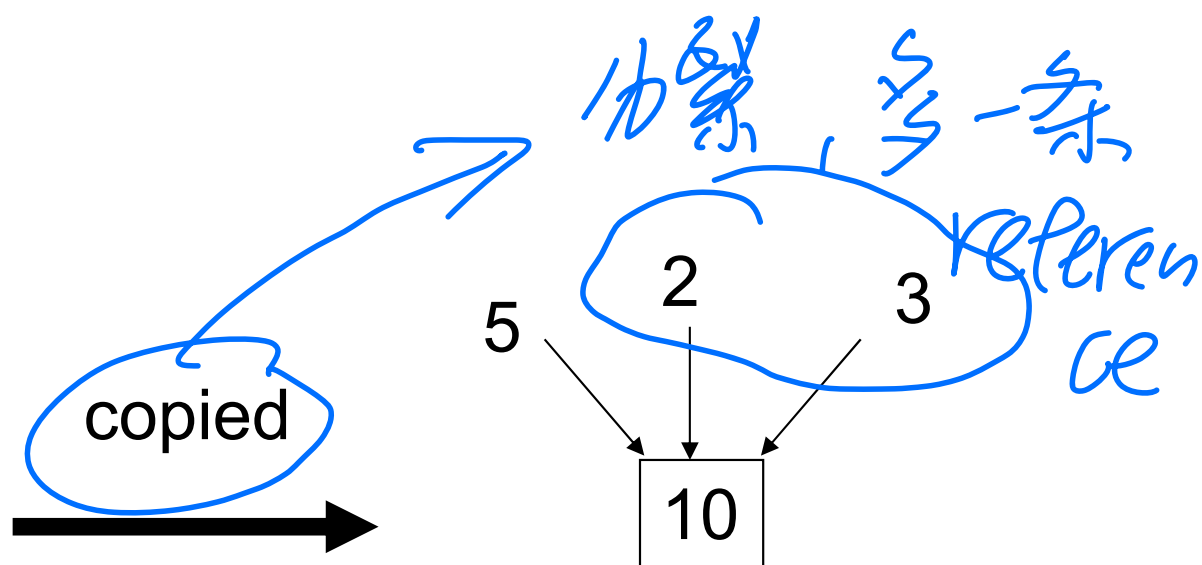
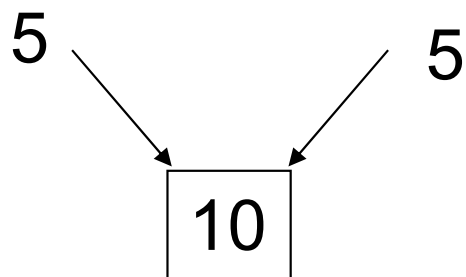






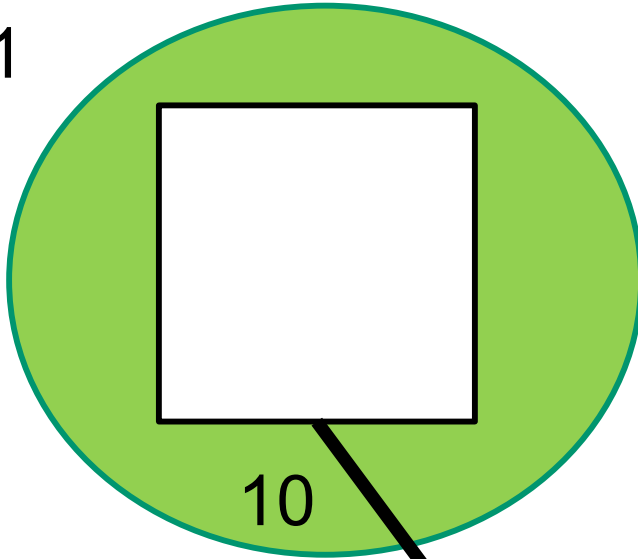
weighted reference counting

- ❑ An **object** has a **weight**
- ❑ assign **weight** to each **pointer**
- ❑ object weight = sum of the pointers' weight
- ❑ when the pointer is **copied**, evenly distribute the weight
- ❑ When a pointer is **deleted**, the weight of the pointer is subtracted from the weight of the object
- ❑ An object is garbage collected when its weight becomes to **0**

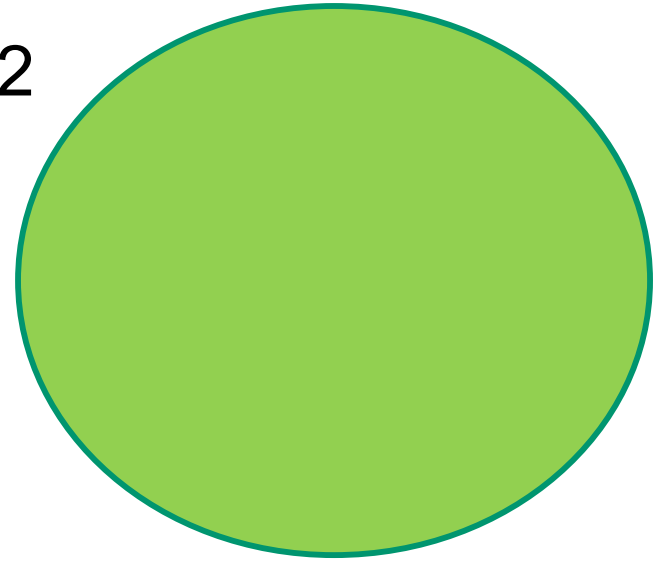


only decrease the weight
no increase the weight

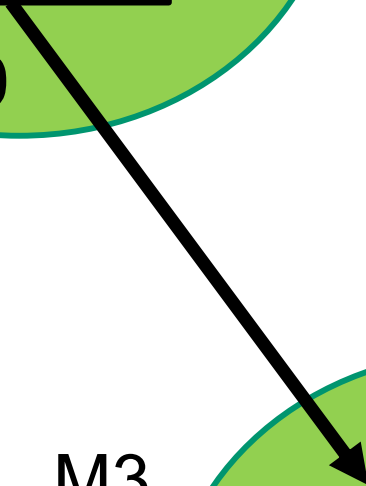
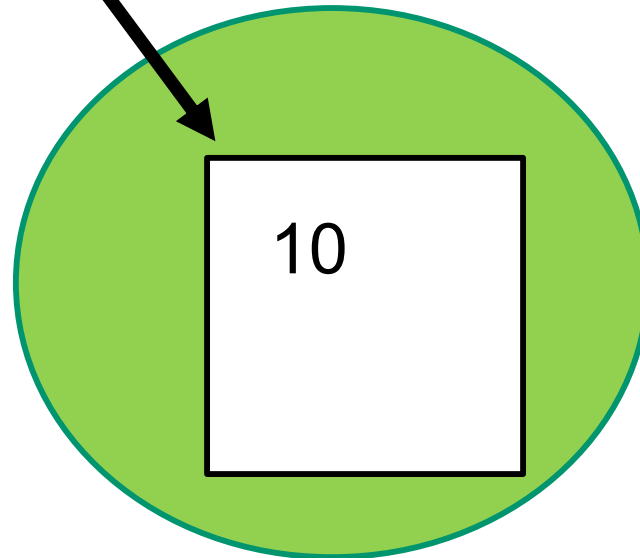
M1

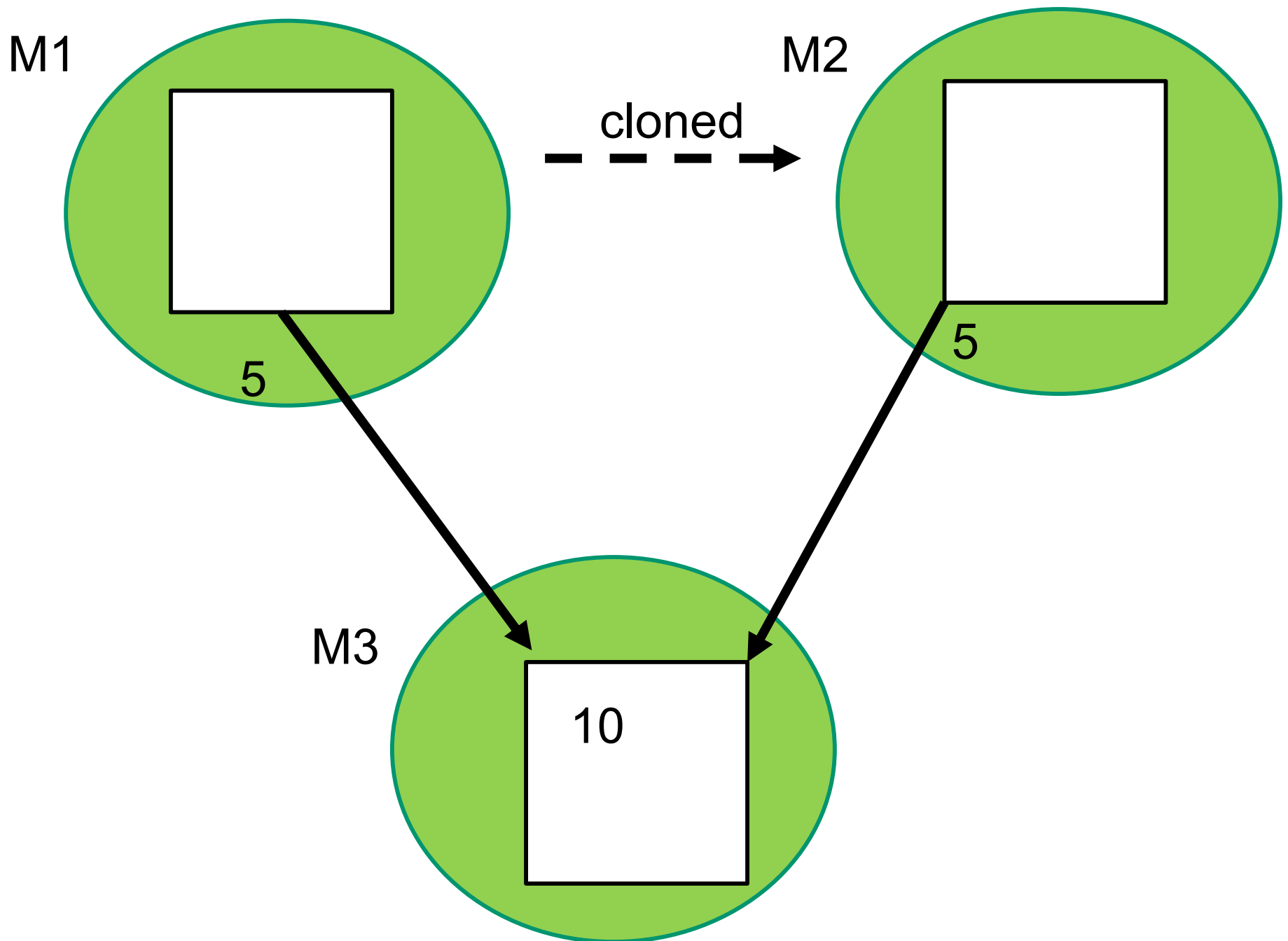


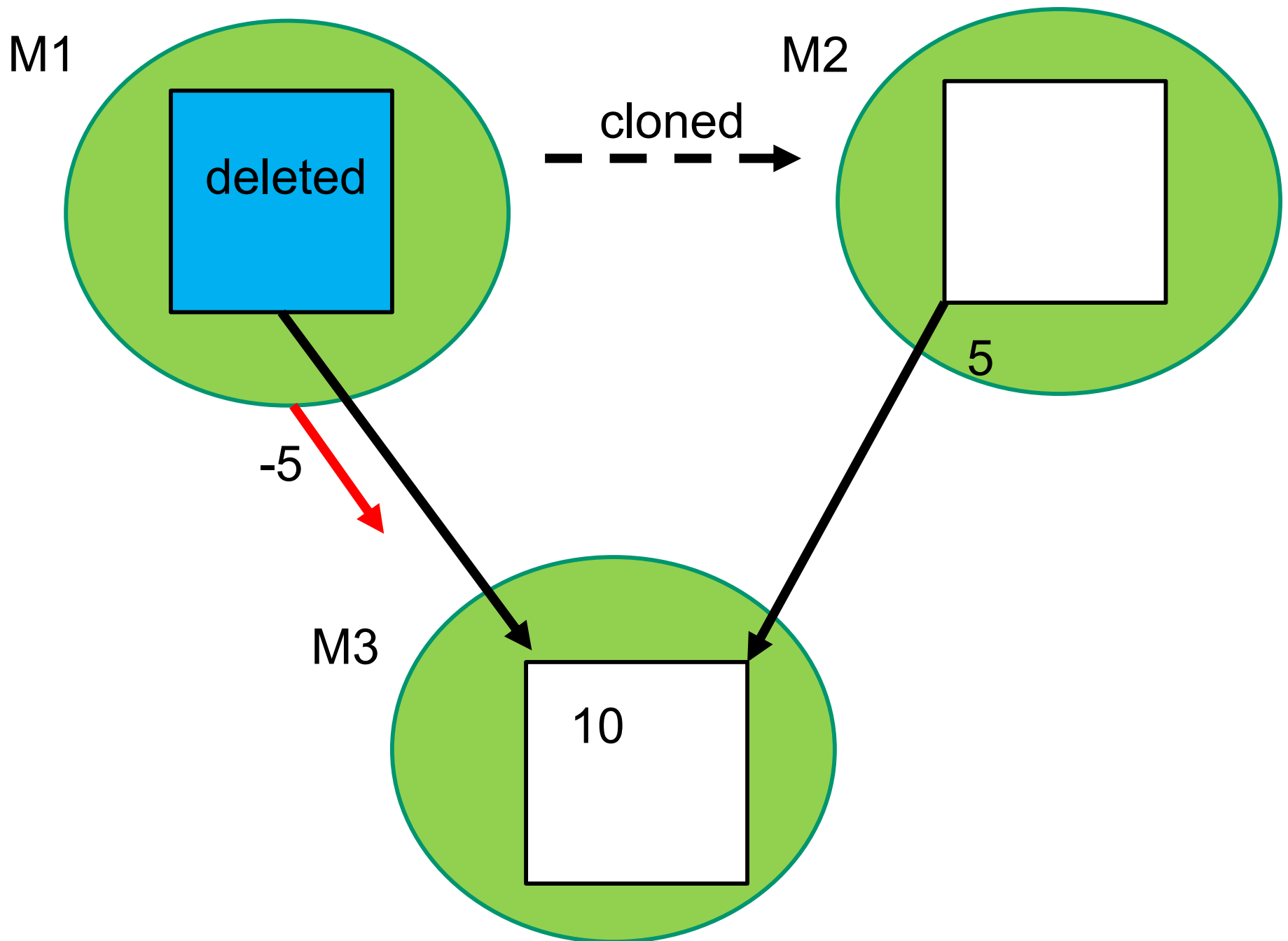
M2



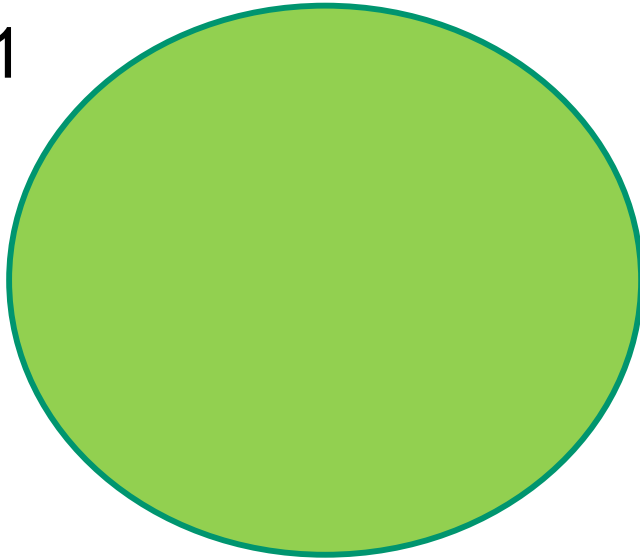
M3







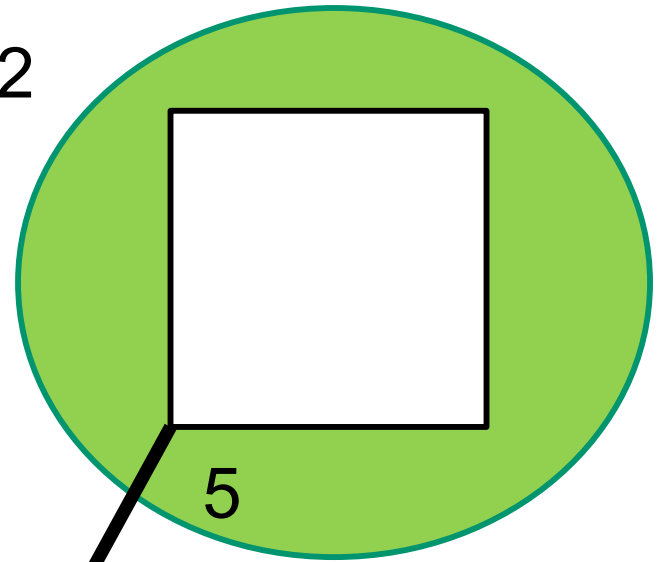
M1



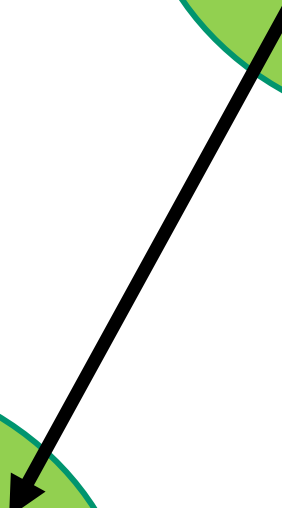
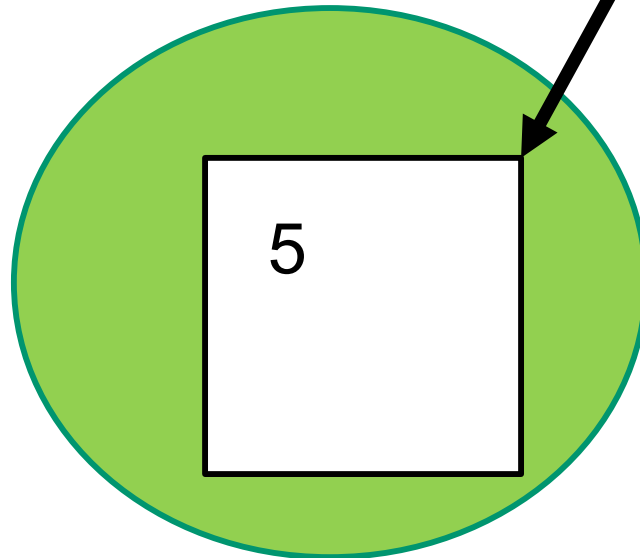
cloned



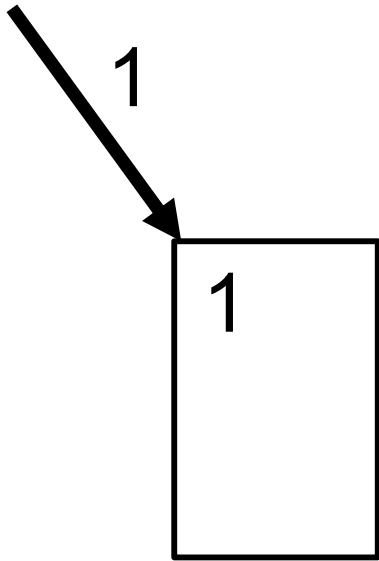
M2

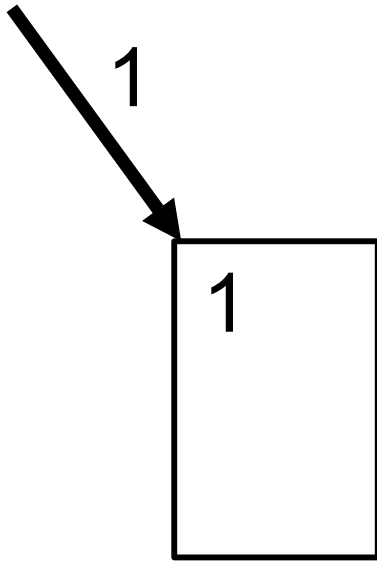


M3

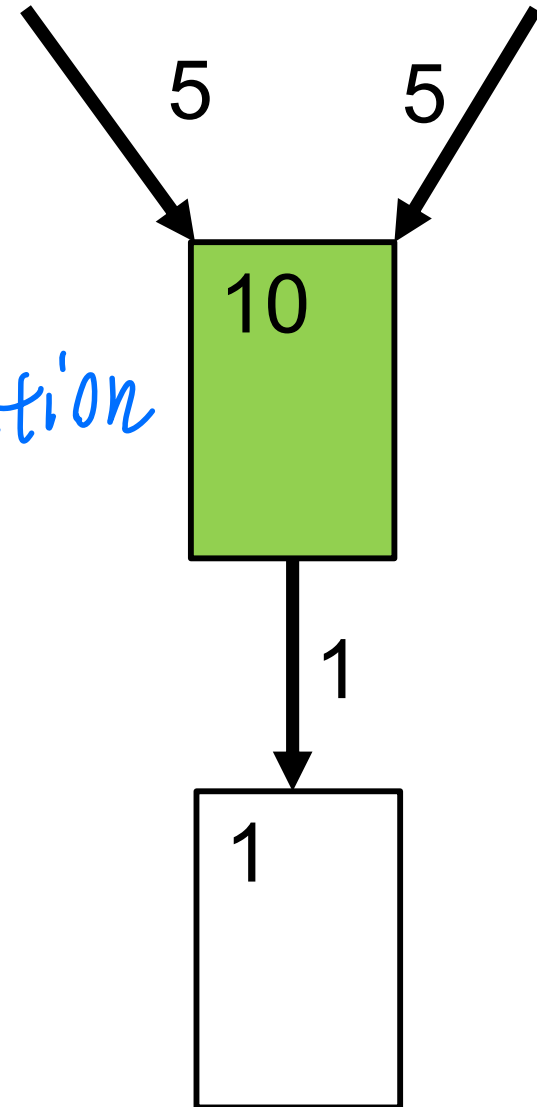


- ❑ The weight in an object only decreases
 - It won't have the problem as the reference counting scheme
- ❑ What if the weight of an object cannot be split?
 - Create an indirection object

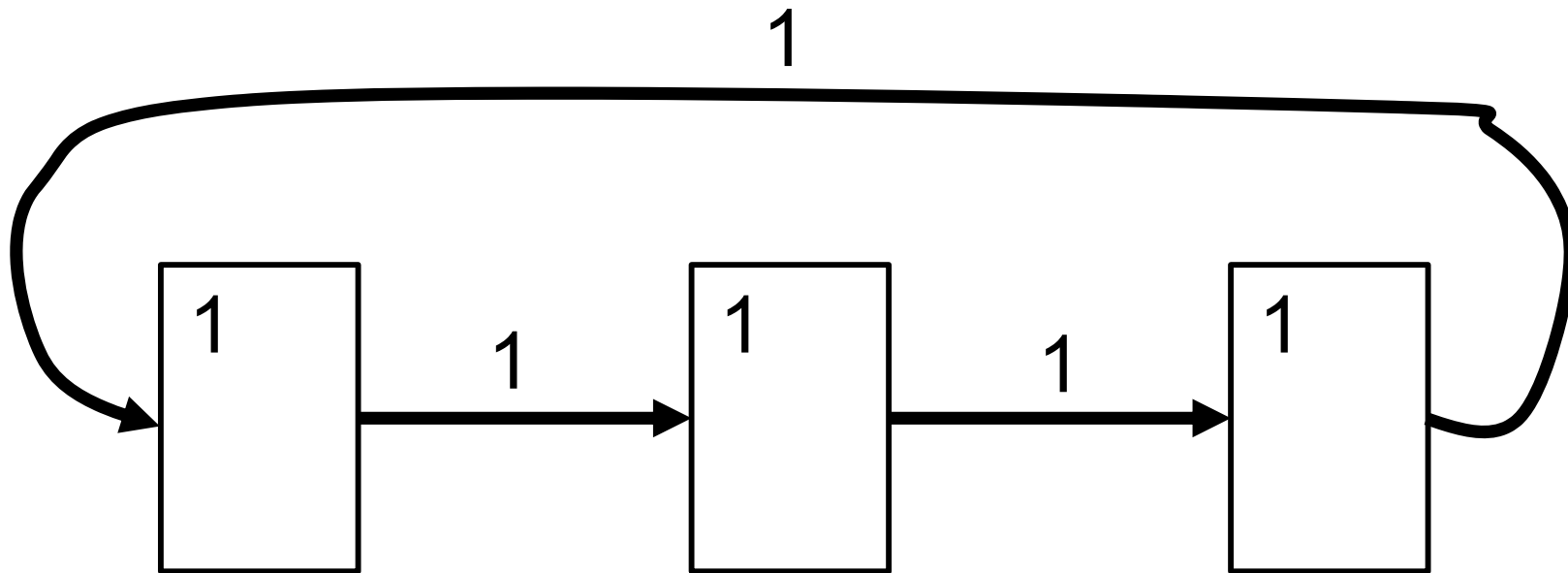




*indirection
objekt*



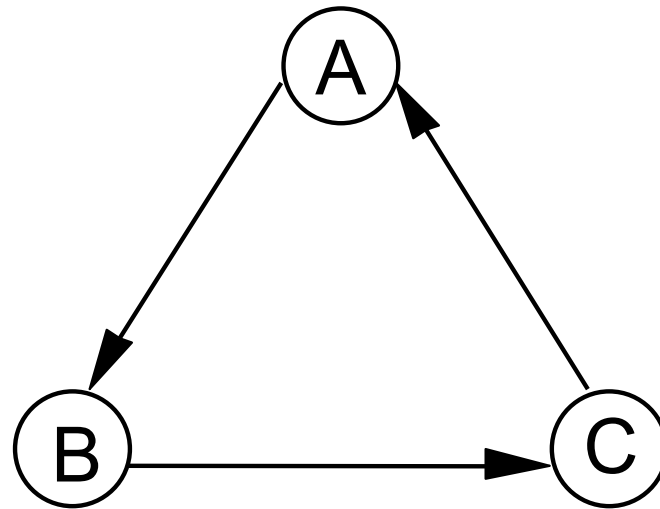
- It cannot handle cyclic garbage



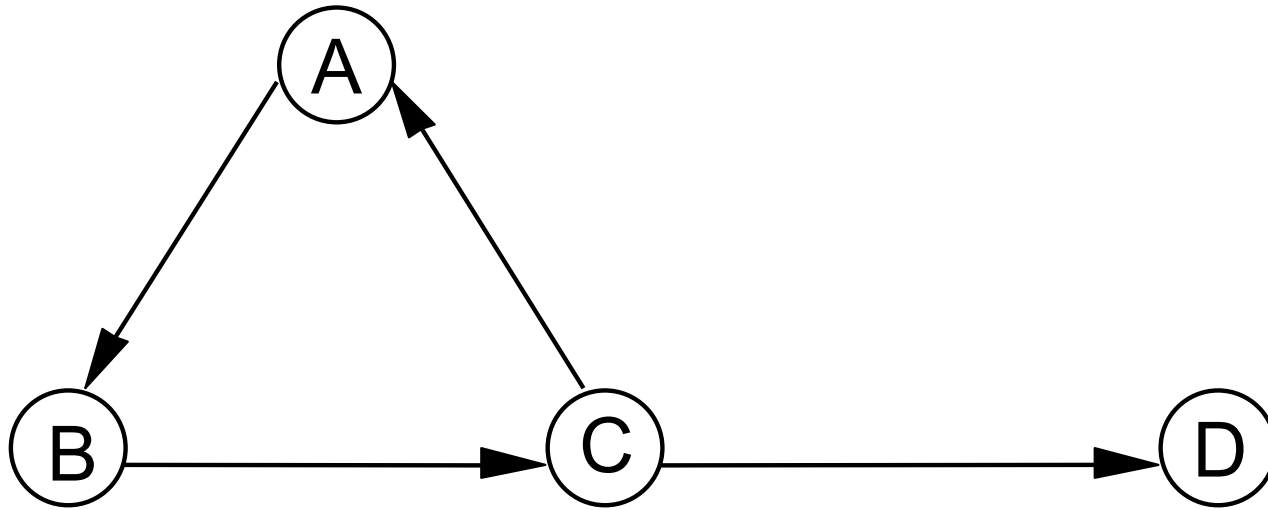
Collecting cyclic garbage

- This works as a complement of other garbage collection algorithms, e.g., weighted reference counting)
- If an object, say A , has a reference to another object, say B , then A is the predecessor of B , and B is A 's successor.
- Let (a) O represent a set of objects, and (b) P be the set of the predecessors of the objects in O . It is easy to see that, if all the objects in O form a cyclic structure, P must be a subset of O .





$O = \{A, B, C\}$
 $P = \{C, A, B\}$

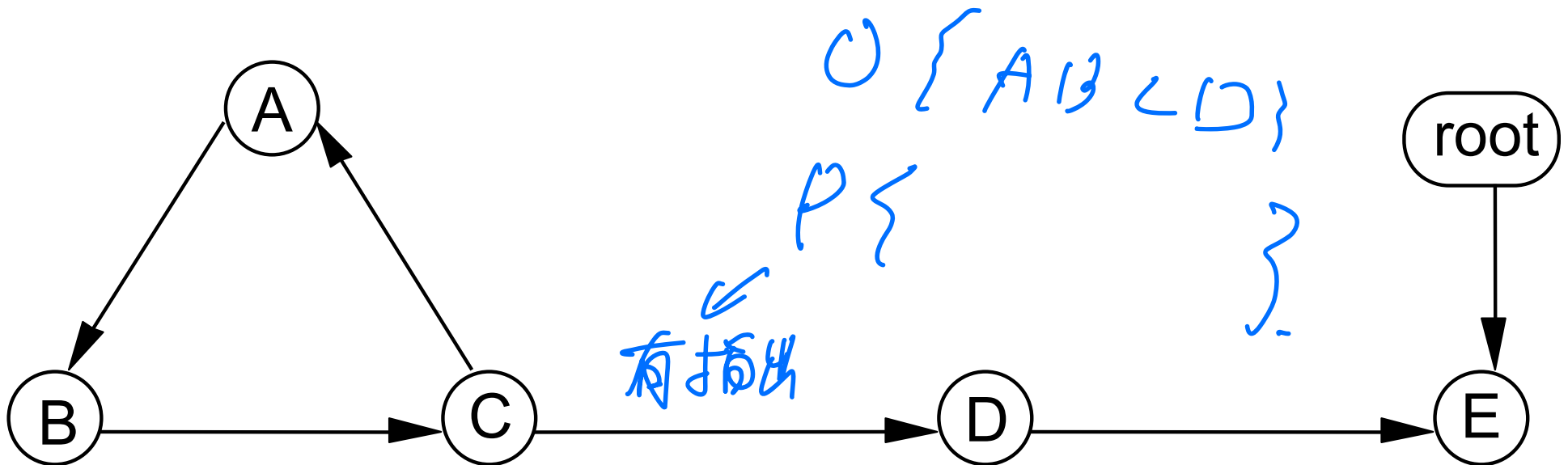


$O = \{A, B, C, D\}$

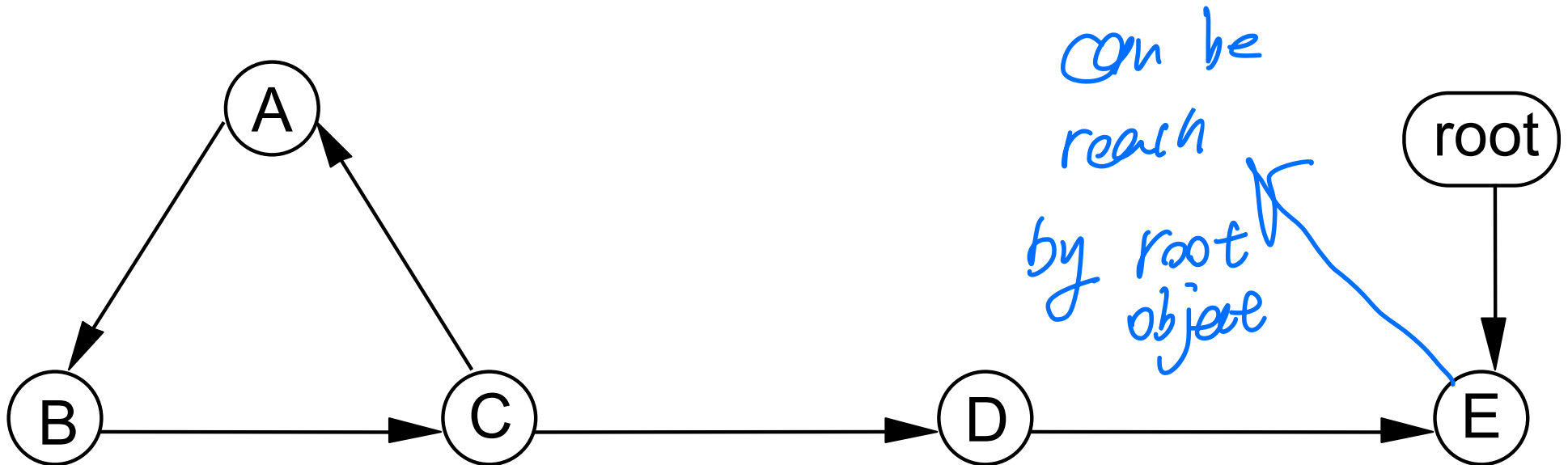
$P = \{C, A, B\}$

The Algorithm

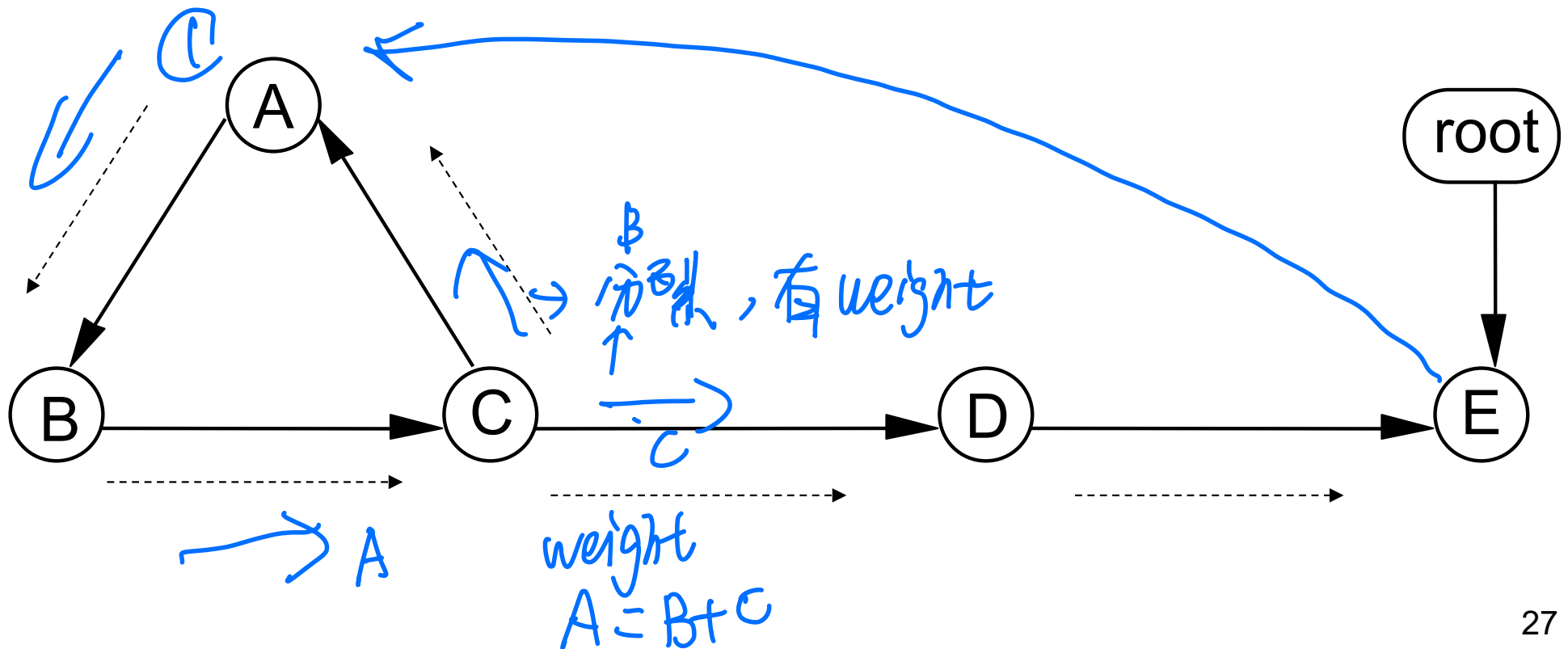
- A **root** object is an operating system object
- Let O be a set of objects which does **not** include any root object, and P be the set containing all the predecessors of the objects in O . If $P \subseteq O$, then objects in O are garbage.



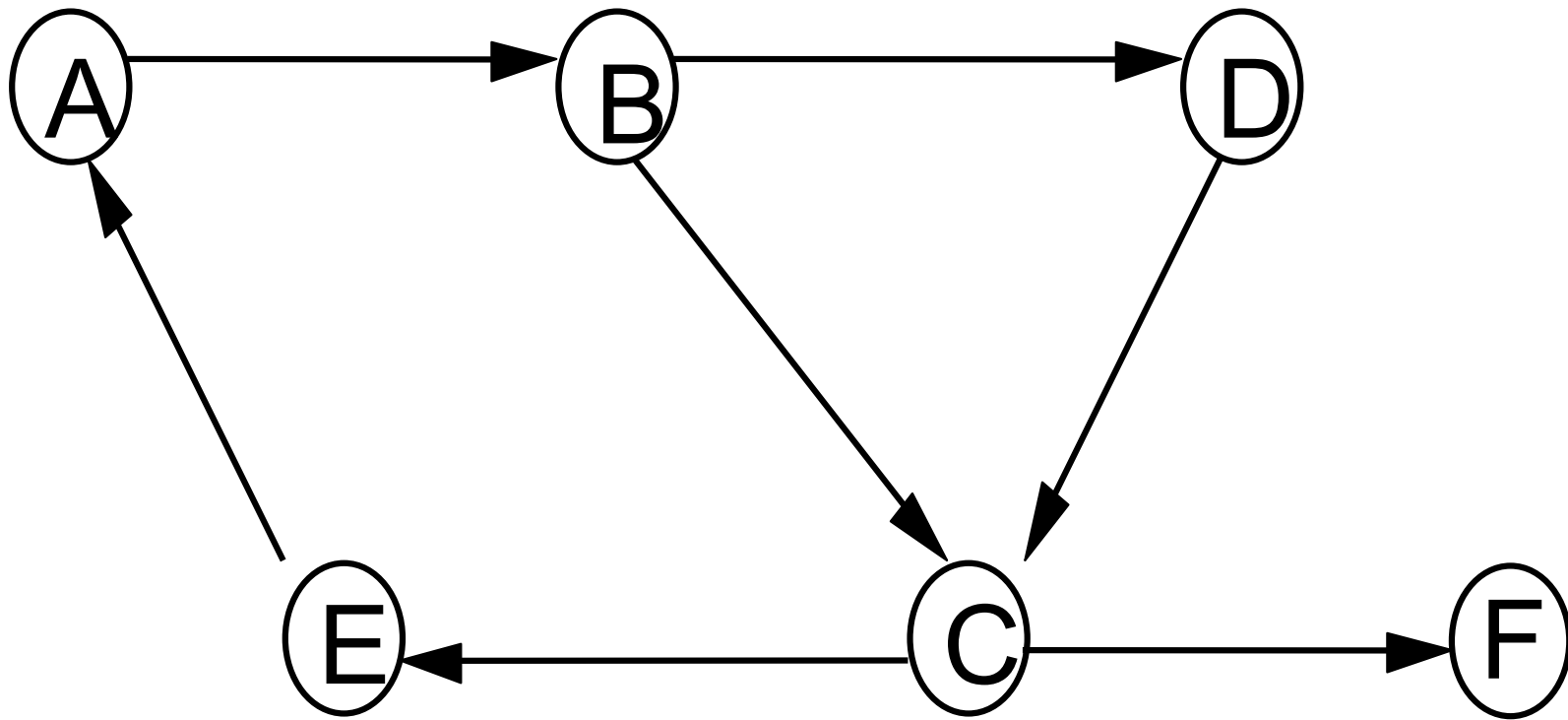
$O = \{A, B, C, D\}$ *garbage*
 $P = \{C, A, B\}$ *include garbage*

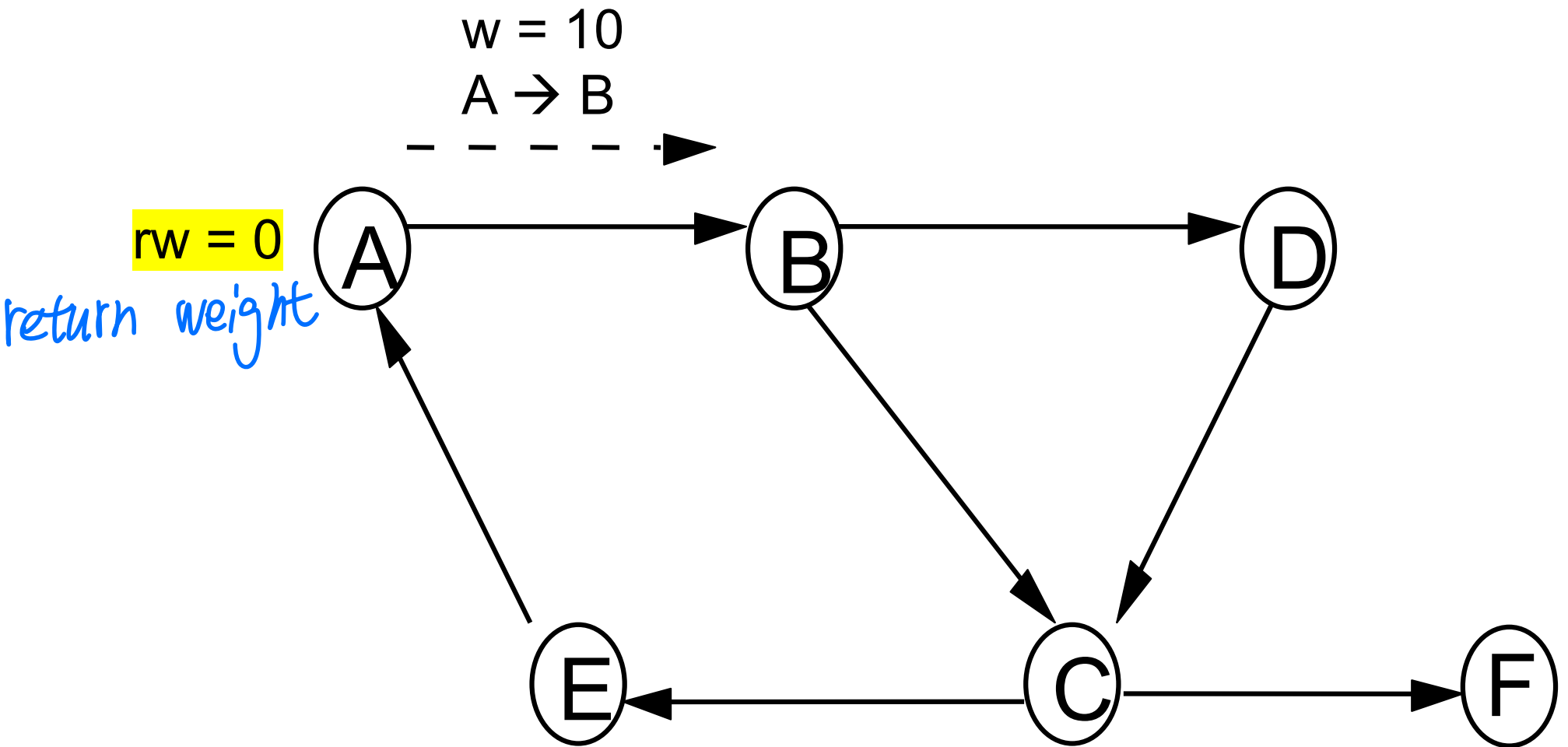


- The algorithm is going to pass some probes to discover the predecessor relationship



- ❑ The probes will be sent back to the initiator of the algorithm
 - How do we know whether we have received all the probes so that we can start looking for cyclic structures?
- ❑ Each probe carries some weight. When a probe p is split into n probes, $p_1, p_2, \dots, p_n$,
 - all the (object, predecessor) pairs in p are copied to p_1 ,
 - $p.w = p_1.w + p_2.w + \dots + p_n.w$,
 - $\sum p_i.\text{initiator} = p.\text{initiator}$ where $1 \leq i \leq n$.
- ❑ How do we know whether we have found all the predecessors of a node?
 - Compare the weight of the node and the sum of the weight of the references to the node (assume the weighted reference counting is used)



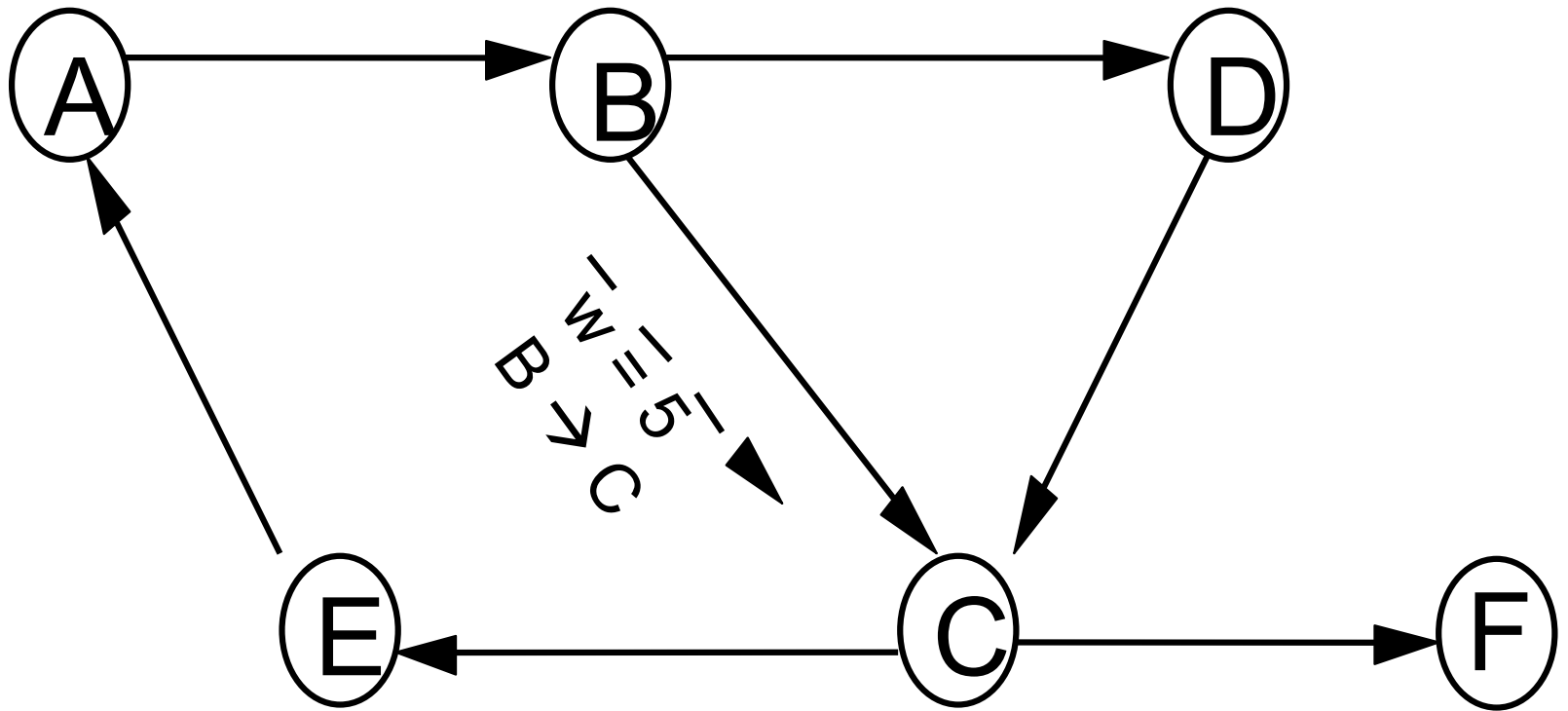


只有一个继承前面的信息

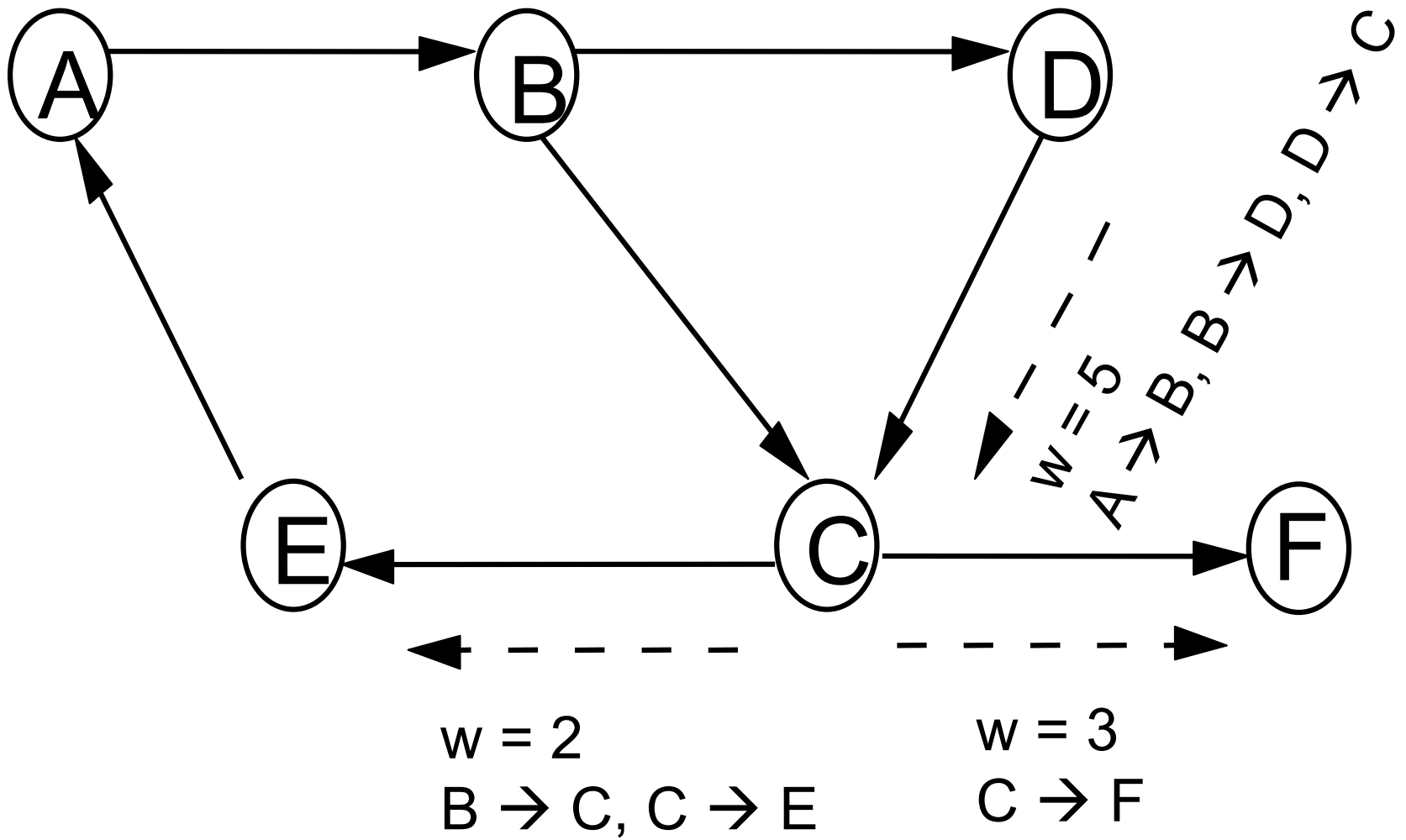
$w = 5$

$A \rightarrow B, B \rightarrow D$

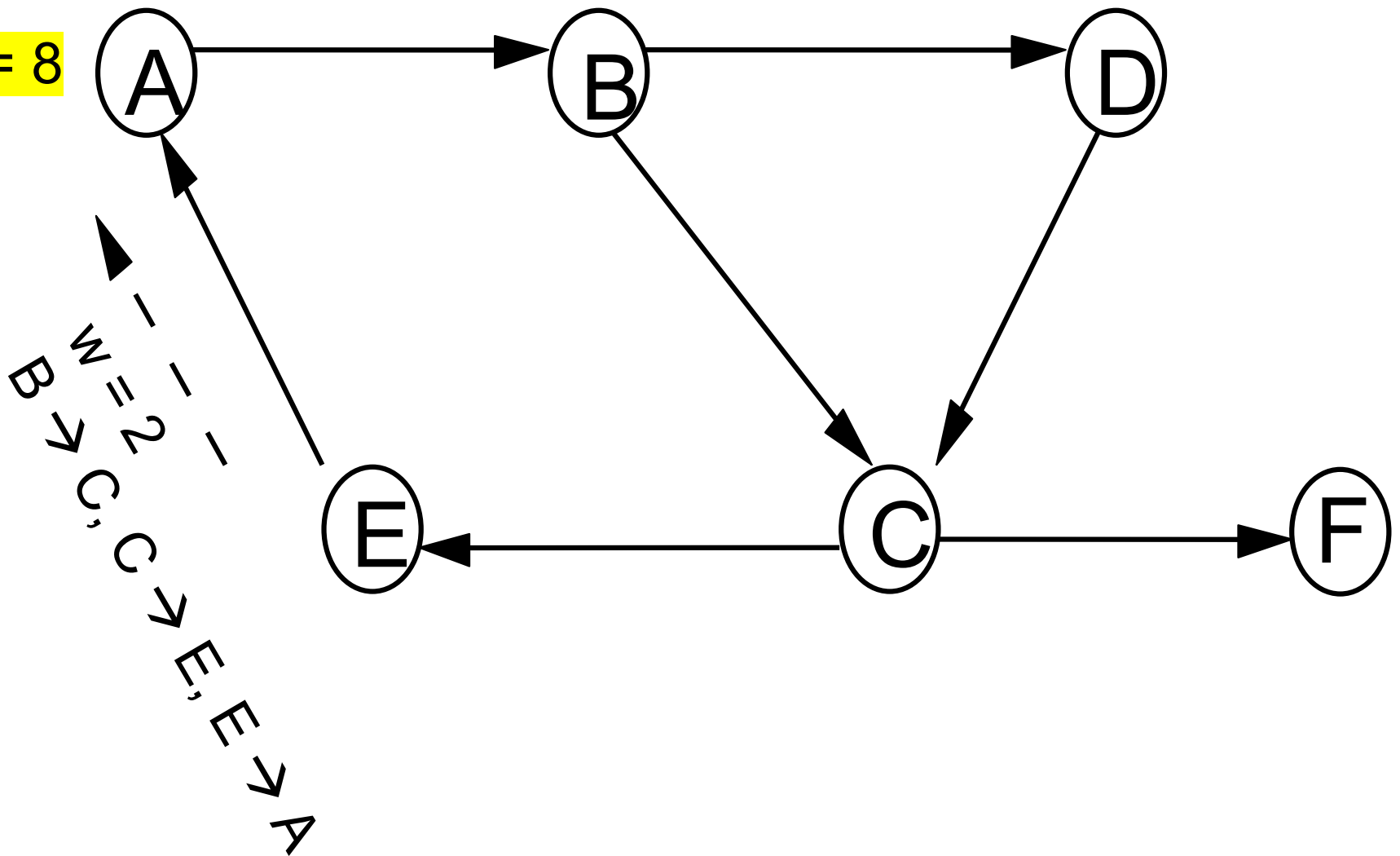
$rw = 0$

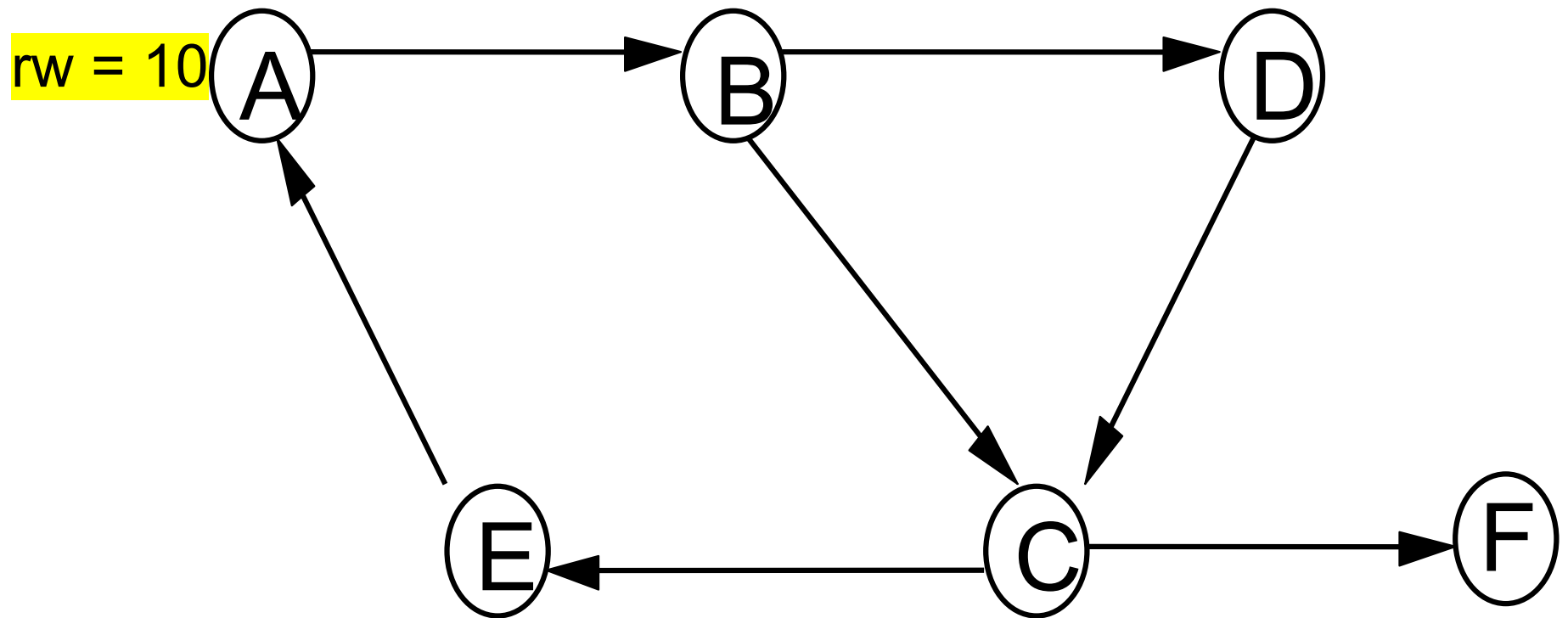


rw = 0



rw = 8





review

- ❑ What problem does the reference counting garbage collection algorithm might have in a distributed system?
- ❑ Describe how the weighted reference counting garbage collection algorithm works.
- ❑ Why does the weighted reference counting garbage collection algorithm not have the problem encountered by the reference counting garbage collection algorithm?
- ❑ Understand how the algorithm for detecting cyclic garbage works.
- ❑ In the algorithm for collecting cyclic garbage, how do we know whether the probes have reached an object through all the predecessors of the object? [hint: the algorithm works as a complement of other garbage collection algorithms, e.g., weighted reference counting]
- ❑ Use the principles of the weighted reference counting to develop an algorithm that finds a set of nodes in a wait-for graph such that the outgoing edges from the nodes in this set always end on nodes in this set. (it would be possible to detect deadlock in the OR deadlock model using this algorithm)

Further reading

- ❑ P.Watson and I.Watson. An efficient garbage collection scheme for parallel computer architectures.PARLE'87, LNCS 259:432-443, Springer Verlag, 1987 (weighted reference counting)
- ❑ X.Ye , J. Keane, Collecting Cyclic Garbage in Distributed Systems, Proceedings of the 1997 International Symposium on Parallel Architectures, Algorithms and Networks